TP3 course validation

The course source is ‘Tp03-2026.pdf‘ in ‘/home/rodrigo/facultad/compi‘.
This validation covers every example and requirement applicable to group 24,
plus the confirmed decisions and regression cases for their interactions.
References are attached to the executable scenarios in
‘tests/tp3_course_cases.py‘; ‘tests/verify_tp3_course.py‘ checks them independently
of the compiler’s exit-status summary.

Coverage by page

TP3 source	Applicable requirement or example	Scenario coverage
p.1	Dynamic RPN, executable statements, references, types and uses	All successful scenarios check sequential addresses, local jump destinations, callable bodies, metadata and absence of residual compile-time operands. ‘large_parameter_list‘ exceeds the initial RPN and parameter storage sizes. The original expression/control cases remain.
p.2	Nested/sibling scopes and mangled identifiers	‘scope_mangling_example‘ implements aa/aaa/bb/cc, checks qualified names and observable shadowing. ‘class_and_function_shadowing‘ covers distinct types and declarations in nested scopes.
p.3	Undeclared, invisible, redeclared symbols and incorrect uses	Separate cases cover variables, functions, classes and methods; forward references, sibling visibility, a later method, cross-category redeclarations, wrong receivers and function-as-destination errors. Both modes must reject with source locations.
p.4	Topic 34 explicit conversions	‘explicit_conversion_example‘ and ‘explicit_conversion_operand‘ evaluate the valid arithmetic forms with TOI. ‘mixed_operands_deferred‘ and ‘float_assignment_deferred‘ preserve the two invalid numeric examples as TP4-boundary cases, without inventing an early compatibility pass.
p.5	Function/method metadata and independent bodies	Formal lists and types are inspected; unused ordinary functions must still have code. Nested, recursive and noncommutative calls, default INTEGER/SINGLEF returns, lexical captures and independent receivers are evaluated.
p.5	Invocation counts, copy semantics and types	Short/long argument and order lists fail. Scalar, object and object-attribute copies are checked for isolation and snapshots before later argument effects. Numeric parameter/return incompatibilities are explicitly compile-only TP4 cases.
p.6	Topic 14 ascending and descending examples	‘from_ascending_example‘ outputs 1 through 9; ‘from_descending_example‘ outputs 10, 8, 6, 4, 2. Equal bounds, overshoot, nested loops, control-variable changes in the body and changes through calls are covered. Negative and zero BY examples make progress through their bodies and verify that no extra sign restriction was introduced.
p.6	Required integer loop operands	‘loop_types_deferred‘ checks that the invalid numeric operand types are preserved for the RPN-to-assembler stage, as directed by p.4. It is not counted as successful runtime type validation.
p.7	Topic 18 embedded identifier/constant assignments	‘embedded_assignment_values‘, ‘operand_snapshot_before_call‘ and ‘comptime_preserves_embedded_effect‘ check assignment values, chains, evaluation snapshots and retained runtime effects.
p.7	Topic 19 examples and evaluation/copy order	The two course results 10 and 8 are checked directly. All six permutations are also checked with a noncommutative result that distinguishes formal positions, with ordinary and compile-time formals. Missing, repeated, invalid and out-of-range positions fail.
p.8, valid 1	Incrementing a compile-time variable and using it	‘comptime_update_example‘ records and substitutes 4$i.
p.8, valid 2	Known selection using a compile-time value	‘comptime_selection_example‘ executes the selected ordinary increment and produces 1.
p.8, valid 3	Known compile-time loop	‘comptime_loop_example‘ traverses 3 through 9 and reaches final value 10, using the group’s FROM construct.
p.8, valid 4	Compile-time formal with a constant actual	‘comptime_parameter_example‘ prints 3 and verifies the default INTEGER return.
p.8, valid 5	Compile-time actual passed to an ordinary formal	‘comptime_to_runtime_parameter‘ prints and returns 3 without changing the caller’s binding.
p.8, invalid 1	Unknown assignment to a compile-time variable	‘comptime_runtime_assignment_error‘ fails at x := z.
p.8, invalid 2	Compile-time modification under unknown control	‘comptime_unknown_control_error‘ checks the same unknown-condition obligation within FROM. Equal writes in both branches and modifications hidden in calls are additional confirmed-rule regressions.
p.8, invalid 3	Unknown actual passed to a compile-time formal	‘comptime_unknown_argument_error‘ fails at the invocation.
p.8, combinations	Compile-time state across control and activations	Known/dead branches, runtime branches, expanded loops, early RET, nested captures, repeated instances, mixed formals, float evaluation, integer limits, uninitialized values, call effects and resource limits are covered.

p.9	Topic 24 class codes and structural compatibility	Renamed fields and different methods remain compatible with matching type/order; count, type, order, qualified class type and code mismatches fail. Effective inherited schemas are compared. Nested and independent copies, including compile-time qualifiers, are evaluated.
p.10	Topic 29 positional existence and visibility	Constant and variable positions, positions supplied at runtime, missing/negative positions, wrong receiver use, compile-time indices and fields declared after the use are checked.
p.11	Topic 32 inherited lookup and ambiguity	Explicit and implicit inherited methods, own method precedence, distinct homonyms, cycles, inherited-first layouts, diamond declaration identity and compatible effective copies are checked.
p.12	Output requirements	Successful output must have main RPN, callable bodies, numbered local jumps, readable literals and semantic TS metadata without debug traces. Errors require source lines and suppress partial RPN. Independent recovery diagnostics are checked.

Explicit adaptations and exclusions

- Generic LONGINT/ULONGINT examples use INTEGER and ‘$i‘, the type assigned to group 24. Program and identifier names follow the group’s lexical rules.
- Calls include the mandatory order list of topic 19, even where a generic example omits it. Missing declarative and executable scaffolding is supplied so each illustration becomes an independent complete program.
- The p.8 generic WHILE examples are adapted to FROM, topic 14. The unknown control rejection example uses an unknown IF inside FROM to exercise the same compile-time modification rule. WHILE syntax is not added to this group.
- The second valid conversion line on p.4 uses ‘toul‘ and describes z as floating, which is inconsistent with this group’s ‘toi‘ and the same page’s mixed-type rule. Its adapted test uses ‘toi(x)+b‘ with INTEGER b. The original spelling is not silently classified as a valid group-24 input.
- Topics 11-13, 17, 20-21, 23, 25-28, 30-31, 33 and 35 belong to other groups. Their alternative language features are outside this compiler’s assignment. Shared function/copy rules still apply where referenced by the assigned topics.
- Simple Reduction is excluded because neither member belongs to the 2011 plan.
- Five numeric incompatibility scenarios remain explicit TP4 boundaries. The suite does not claim that their assembler or eventual runtime behavior has been implemented or validated.

Checks and results

The original verifier runs 107 checks. The course verifier defines 170 scenarios and compiles each in both modes, giving 340 additional compiler runs:

- 103 valid scenarios have their RPN evaluated against explicit observable output.
- 60 negative scenarios must fail with the required diagnostic and no partial RPN.
- Seven scenarios are compile-only: five TP4 boundaries and two intentionally nonterminating runtime programs that must not be expanded during compilation.

Successful IR also receives structural and metadata checks. Jump destinations are local and valid, every CALL has a body, declaration identities are unique in effective schemas, formal lists reference parameters, and compile-time reads and ARG stores do not survive as runtime operands. MATERIALIZE snapshots must refer to declared compile-time fields and numeric constants.

The observable evaluator implements INTEGER division, SINGLEF rounding, lexical parents, pending call frames, object receivers and value copies. It does not execute the compiler’s specialization algorithm to obtain its expected values. Output expectations are explicit, or calculated from an independent evaluation of source argument order. No production failure was resolved by changing the expected source behavior to match the compiler’s incorrect output.

The plain-build per-scenario record, including the compiler SHA-256, is in tp3_test_results.json. The instrumented binary’s matching scenario report is in tp3_test_results_asan.json. All 340 course runs pass in each report. The original 107 checks also pass on both binaries, for 447 compiler checks per build. Both builds have zero Bison conflicts and compile with warnings treated as errors.

Defects found and corrected

1. Cached calls restored unrelated compile-time bindings from their original environment. Effect replay now updates only actual effects; repeated calls inside expanded loops no longer roll the caller’s counter backward.
2. Known nonlocal runtime writes were discarded after a call. Proven final values are propagated, so a function that sets the FROM control variable to a known bound does not make the compile-time loop spuriously unknown.
3. Indexed writes and whole-object copies involving compile-time fields were missing from call-effect summaries. Summaries now include them, preserving updates, driving loop expansion and preventing a cached call from bypassing the unknown-control rule.
4. Object argument copies lost compile-time fields or read them after subsequent arguments had changed the source. Bindings are captured at the argument copy, and separate activation environments receive the captured field values.
5. A compile-time source field had no runtime value available when copied into a compatible ordinary field. MATERIALIZE retains a logical object value with constant-field snapshots in TS metadata, while runtime destinations continue to omit compile-time storage.

The tests also exposed limitations in the original oracle: local-class methods needed the correct enclosing lexical frame; integer division and floating-point rounding needed the assigned representations; literal/symbol category collisions needed separate interpretation; and object copies needed the effective schema. Those oracle corrections remain distinct from compiler corrections.

Reproduction

```bash
./run_tests.sh
python3 tests/verify_tp3_course.py --case comptime_loop
python3 tests/verify_tp3_course.py --report docs/tp3_test_results.json
```

Both suites are also run with ASan, UBSan and leak detection. In this environment Clang supplies the installed sanitizer runtime, and LeakSanitizer requires running outside the restricted sandbox to inspect process state.